



Especificação do Projeto: Pedra-Papel-Tesoura Fase 4 (Arquitetura Cliente-Servidor com Java RMI)

1. Contextualização e Objetivo

Nas fases anteriores, vocês dominaram a concorrência e a infraestrutura de rede manipulando Sockets e fluxos de bytes. Nesta **Fase 4**, daremos um salto em direção a uma **Arquitetura Cliente-Servidor** pura e focada no domínio do problema, utilizando o padrão **RPC (Remote Procedure Call)** através de sua implementação nativa em Java: o **RMI (Remote Method Invocation)**. O grande paradigma aqui é a centralização: o Servidor deixa de ser um mero repassador de mensagens e torna-se o guardião absoluto do estado do jogo. Os clientes, por sua vez, tornam-se terminais "leves" que não processam regras de negócio; eles apenas invocam métodos de um objeto que reside remotamente na memória do servidor, abstraindo completamente a complexidade da rede.

2. O Servidor como uma Máquina de Estados

Ao utilizarmos RMI, a própria JVM (*Java Virtual Machine*) passa a gerenciar o *pool* de *Threads* que atende as requisições de rede. Isso significa que vocês **não precisam mais instanciar Threads manualmente** (`extends Thread`) ou usar `wait()`/`notify()` para os jogadores.

Sem os laços infinitos travando as threads locais, o Servidor se transforma em uma **Máquina de Estados Reativa**. Ele fica passivo na memória e muda seu comportamento de acordo com os estímulos recebidos pelos clientes remotos.

A arquitetura do servidor deve transitar logicamente entre os seguintes estados:

1. **ESPERANDO_JOGADORES**: O servidor aceita registros. Ao atingir o limite da sala (ex: 2 jogadores), ele inicializa a **Mesa** e avança.
2. **FASE_APOSTA**: O servidor apenas recebe e guarda as apostas. Quando *todos* os jogadores ativos enviarem suas apostas, ele processa os descontos e avança para a próxima fase.
3. **FASE_JOGADA**: O servidor recebe e guarda as escolhas (Pedra, Papel ou Tesoura). Ao receber todas, executa a lógica de resolução. Se houver vencedor, paga o prêmio e volta para **FASE_APOSTA**. Se houver empate, avança para o desempate.
4. **FASE_DESEMPATE**: Aguarda a decisão dos jogadores (Continuar ou Desistir). Processa o W.O. ou as apostas extras e retorna para a **FASE_JOGADA**.

Nota de Segurança: Como o RMI atende múltiplos clientes simultaneamente, é estritamente necessário utilizar a palavra-chave `synchronized` nos métodos do servidor que alteram essa máquina de estados, para evitar condições de corrida.

3. O Papel do Cliente e a Observabilidade

No modelo RMI, o código do cliente fica extremamente enxuto. O Cliente não possui a classe **Mesa** e não possui a classe **Jogo**. Ele apenas se conecta ao diretório remoto (Registry), resgata o *Stub* (a interface) e passa a fazer requisições.

A responsabilidade do cliente é implementar um mecanismo de **Polling** (perguntar periodicamente ao servidor "em qual fase estamos?") e reagir de acordo.

Requisito de Observabilidade: Em sistemas distribuídos, a falta de *logs* cria "caixas pretas". O cliente deve ser verboso. Ele deve imprimir obrigatoriamente no console do usuário tudo o que

está fazendo em cada etapa da máquina de estados:

- Imprimir quando se conecta e qual ID recebeu.
- Imprimir quando o servidor avança de fase.
- Imprimir o valor que decidiu apostar antes de enviá-lo pela rede.
- Imprimir a jogada que escolheu antes de chamar o método remoto.
- Capturar a `RemoteException` e imprimir uma mensagem amigável caso o servidor saia do ar (ex: "Erro: Conexão com o servidor perdida.").

4. Sugestão de Contrato (Interface RMI)

Para garantir que clientes e servidores falem a mesma língua, vocês devem criar uma interface que herde de `java.rmi.Remote`. O Servidor implementará esta interface, e o Cliente a utilizará para fazer as chamadas.

Abaixo está a sugestão do contrato base. Notem o uso do objeto genérico `MensagemJogo` (criado na Fase 3) para padronizar o tráfego de dados.

Java

```
import java.rmi.Remote;
import java.rmi.RemoteException;

// Este arquivo deve estar presente tanto no projeto do Servidor quanto no do
// Cliente
public interface ContratoJogo extends Remote {

    // Método para o cliente entrar na partida e receber um ID único do servidor
    int registrarJogador(String nome) throws RemoteException;

    // Método utilizado para o Polling do cliente.
    // O servidor deve retornar qual é a Fase Atual do jogo.
    MensagemJogo aguardarSuaVez(int idJogador) throws RemoteException;

    // Método único para o cliente enviar suas ações.
    // A 'acao' pode conter uma Apostas, uma Jogada (Pedra/Papel/Tesoura) ou
    // Decisão de Desempate.
    void enviarAcao(int idJogador, MensagemJogo acao) throws RemoteException;
}
```